

Blockchains & Cryptocurrencies

Smart Contracts / Ethereum In Detail (II)



Instructor: Matthew Green
Spring 2026

🛡️ Exploit Analysis: Truebit Protocol (~\$26M Loss) @Truebitprotocol A reminder that one missed check is all it takes.

As shown in the image, the purchase function utilized SafeMath for most operations but missed the final addition step.

The Vulnerability: The missing safe-add allowed an integer overflow when a large enough mint amount was passed, resulting in a token cost of zero.

```
function 0x1446(uint256 varg0) private {  
    require(bool(stor_97_0_19.code.size));  
    v0, /* uint256 */ v1 = stor_97_0_19.totalSupply().gas(msg.gas);  
    require(bool(v0), 0, RETURNDATASIZE()); // checks call status, pr  
    require(RETURNDATASIZE() >= 32);  
    v2 = 0x18ef(v1, v1); safe mul/sub/div  
    v3 = 0x18ef(_setParameters, v2);  
    v4 = 0x18ef(v1, v1);  
    v5 = 0x18ef(100, v4);  
    v6 = _SafeSub(v3, v5);  
    v7 = 0x18ef(varg0, _reserve);  
    v8 = 0x18ef(v1, v7);  
    v9 = 0x18ef(200, v8);  
    v10 = 0x18ef(varg0, _reserve);  
    v11 = 0x18ef(varg0, v10);  
    v12 = 0x18ef(100, v11);  
    v13 = _SafeDiv(v6, v12 + v9);  
    return v13;  
}
```


News?

Truebit hit by \$26m exploit as attackers increasingly target older DeFi protocols



Home > DeFi > Truebit hit by \$26m exploit as attackers increasingly target older DeFi protocols

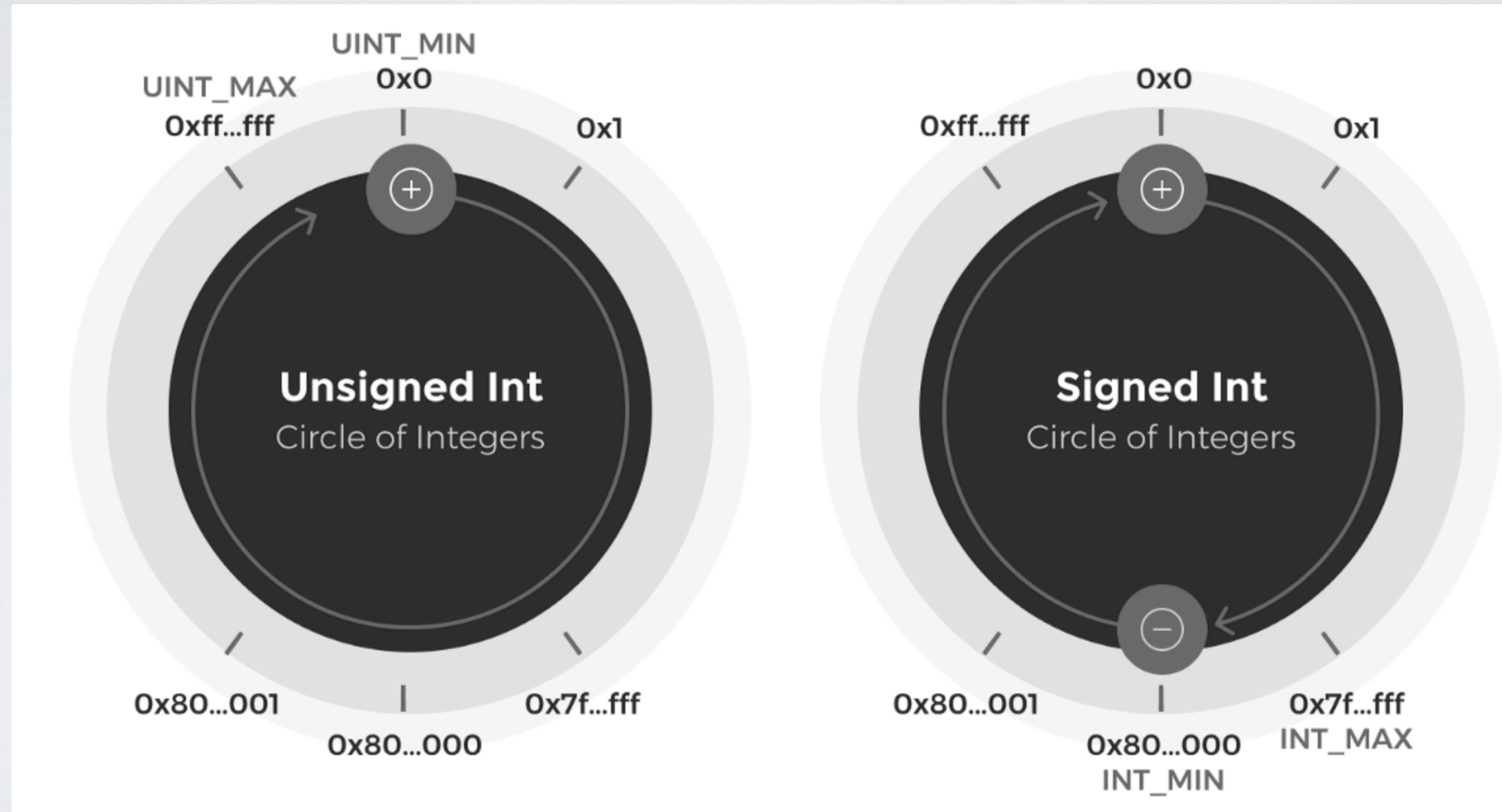
By **Tim Craig**

9 January 2026 at 12:06

Share

- Truebit suffers a \$26 million exploit.
- It comes as attacks targeting older DeFi protocols increase.

News?



Concurrency and re-entrancy

- Ethereum transactions run sequentially and atomically
- In principle this is good: there **appear** to be no concurrency issues (no threads) and your methods always run to completion or don't complete at all!

Example (ok)

```
function transfer(uint amount, address recipient) ... {  
    if (balances[msg.sender] < amount) {  
        // insufficient balance  
        revert("Something bad happened");  
    }  
  
    // Transfer the money  
    balances[msg.sender] -= amount;  
    balances[recipient] += amount;  
}
```

Re-entrancy

```
function transferAndNotify(uint amount, address notifyContract) ... {  
    if (balances[msg.sender] < amount) {  
        // insufficient balance  
        revert("Something bad happened");  
    }  
  
    // Call the specified contract to notify it that a deposit is coming  
    notifyContract.notify(amount, "You are getting a deposit!");  
  
    // Transfer the money  
    balances[msg.sender] -= amount;  
    balances[notifyContract] += amount;  
}
```


Re-entrancy

- There is still one scary “gotcha”!
- Ethereum is not re-entrancy “safe”. *You can still have multiple calls to the same routine within any given call-stack.*

```
contractA.bar()
```

```
contractB.foo()
```

```
contractA.bar()
```


Re-entrancy

```
function transferAndNotify(uint amount, address notifyContract) ... {  
    if (balances[msg.sender] < amount) {  
        // insufficient balance  
        revert("Something bad happened");  
    }  
    balances[notifyContract] += amount;  
  
    // Call the specified contract to notify it that a deposit is coming  
    notifyContract.notify(amount, "You are getting a deposit!");  
  
    // Transfer the money  
    balances[msg.sender] -= amount;  
}
```

Re-entrancy

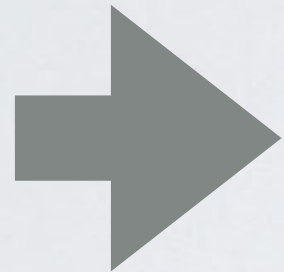
```
function transferAndNotify(uint amount, address notifyContract) ... {  
    if (balances[msg.sender] < amount) {  
        // insufficient balance  
        revert("Something bad happened");  
    }  
    balances[notifyContract] += amount;  
  
    // Call the specified contract to notify it that a deposit is coming  
    notifyContract.notify(amount, "You are getting a deposit!");  
  
    // Transfer the money  
    balances[msg.sender] -= amount;  
}
```

What if this contract call
calls us?

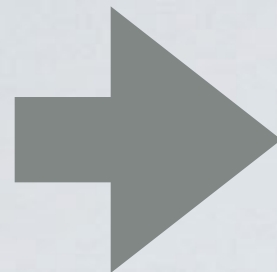
Re-entrancy

```
function transferAndNotify(uint amount, address notifyContract) ... {  
    if (balances[msg.sender] < amount) {  
        // insufficient balance  
        revert("Something bad happened");  
    }  
    balances[notifyContract] += amount;  
  
    // Call the specified contract to notify it that a deposit is coming  
    notifyContract.notify(amount, "You are getting a deposit!");  
  
    // Transfer the money  
    balances[msg.sender] -= amount;  
}
```

What if this contract call
calls us?



Re-entrancy

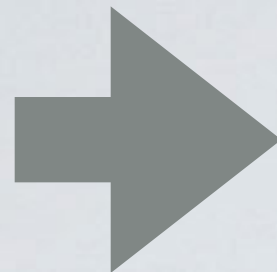


```
function transferAndNotify(uint amount, address notifyContract) ... {  
    if (balances[msg.sender] < amount) {  
        // insufficient balance  
        revert("Something bad happened");  
    }  
    balances[notifyContract] += amount;  
  
    // Call the specified contract to notify it that a deposit is coming  
    notifyContract.notify(amount, "You are getting a deposit!");  
  
    // Transfer the money  
    balances[msg.sender] -= amount;  
}
```



What if this contract call
calls us?

Re-entrancy



```
function transferAndNotify(uint amount, address notifyContract) ... {  
    if (balances[msg.sender] < amount) {  
        // insufficient balance  
        revert("Something bad happened");  
    }  
    balances[notifyContract] += amount;  
  
    // Call the specified contract to notify it that a deposit is coming  
    notifyContract.notify(amount, "You are getting a deposit!");  
  
    // Transfer the money  
    balances[msg.sender] -= amount;  
}
```



What if this contract call
calls us?

“The DAO”

```
function splitDAO(
    uint _proposalID,
    address _newCurator
) noEther onlyTokenholders returns (bool _success) {

    ...
    // XXXXX Move ether and assign new Tokens. Notice how this is done first!
    uint fundsToBeMoved =
        (balances[msg.sender] * p.splitData[0].splitBalance) /
        p.splitData[0].totalSupply;
    if (p.splitData[0].newDAO.createTokenProxy.value(fundsToBeMoved)(msg.sender)
== false) // XXXXX This is the line the attacker wants to run more than once
        throw;

    ...
    // Burn DAO Tokens
    Transfer(msg.sender, 0, balances[msg.sender]);
    withdrawRewardFor(msg.sender); // be nice, and get his rewards
    // XXXXX Notice the preceding line is critically before the next few
    totalSupply -= balances[msg.sender]; // XXXXX AND THIS IS DONE LAST
    balances[msg.sender] = 0; // XXXXX AND THIS IS DONE LAST TOO
    paidOut[msg.sender] = 0;
    return true;
}
```


Solutions?

```
function transferAndNotify(uint amount, address notifyContract) ... {  
    if (globalLock == true) {  
        revert("Locked.");  
    }  
    globalLock = true;  
  
    if (balances[msg.sender] < amount) {  
        // insufficient balance  
        revert('Something bad happened');  
    }  
  
    // Call the specified contract to notify it that a deposit is coming  
    notifyContract.notify(amount, "You are getting a deposit!");  
  
    // Transfer the money  
    balances[msg.sender] -= amount;  
    balances[notifyContract] += amount;  
  
    globalLock=false;  
}
```



```

function transferAndNotify(uint amount, address notifyContract) ... {
    if (globalLock == true) {
        revert("Locked.");
    }
    globalLock =
    if (balances[msg.sender] < amount) {
        // insufficient balance
        revert("Something went wrong");
    }

    // Call the specific notify function
    notifyContract.notify(msg.sender, amount);

    // Transfer the money
    balances[msg.sender] -= amount;
    balances[notifyContract] += amount;

    globalLock=false;
}

```

Drawbacks of (global) locks:

1. Extra gas (due to stores/loads)
2. Can get “stuck” if you’re careless
3. Sometimes re-entrant calls are useful!

Check-Effects-Interaction pattern

- Most common solution is to follow a code pattern:
 - First perform all contract checks (CHECKS)
 - Second, update contract state (EFFECTS)
 - Finally, make any contract calls (INTERACTION)

Check-Effects-Interaction pattern

```
function transferAndNotify(uint amount, address notifyContract) ... {  
  // CHECK  
  require (amount < balances[msg.sender]);  
  
  // EFFECTS  
  balances[msg.sender] -= amount;  
  balances[notifyContract] += amount;  
  
  // INTERACTION  
  notifyContract.notify(amount, "You are getting a deposit!");  
}
```

Contract upgrades

- Ethereum contracts are not (natively) upgradeable
 - Once a contract is deployed, it can self-destruct
 - But its code cannot be changed
 - But some contracts need to be upgraded (bug fixes, etc.)
 - How are we going to handle this?